

6.3 TileAndDistributeToWorkgroups pass 工作流程

设备级分块过程输入 IR 分析

```
%cst = arith.constant 0.000000e+00 : f16
%c0 = arith.constant 0 : index
%0 = hal.interface.binding.subspan ... : !flow.dispatch.tensor<readonly:tensor<512x512xf16>>
%1 = hal.interface.binding.subspan ... : !flow.dispatch.tensor<readonly:tensor<512x512xf16>>
%2 = hal.interface.binding.subspan ... : !flow.dispatch.tensor<writeonly:tensor<512x512xf16>>
%3 = flow.dispatch.tensor.load %0, offsets = [0, 0], sizes = [512, 512], strides = [1, 1] : ...tensor<512x512xf16>> -> tensor<512x512xf16>
%4 = flow.dispatch.tensor.load %1, offsets = [0, 0], sizes = [512, 512], strides = [1, 1] : ...tensor<512x512xf16>> -> tensor<512x512xf16>
%5 = tensor.empty() : tensor<512x512xf16>
%6 = linalg.fill {lowering_config = #iree_codegen.lowering_config<tile_sizes = [[32, 32, 32]]>} ins(%cst : f16) outs(%5 :
tensor<512x512xf16>) -> tensor<512x512xf16>
%7 = linalg.matmul {lowering_config = #iree_codegen.lowering_config<tile_sizes = [[32, 32, 32]]>} ins(%3, %4 : tensor<512x512xf16>,
tensor<512x512xf16>) outs(%6 : tensor<512x512xf16>) -> tensor<512x512xf16>
flow.dispatch.tensor.store %7, %2, offsets = [0, 0], sizes = [512, 512], strides = [1, 1] : tensor<512x512xf16> ->
!flow.dispatch.tensor<writeonly:tensor<512x512xf16>>
```

设备级分块过程实现

1. 循环分布方法 (distributionMethod)
2. 获取计算操作
3. 获取分块和循环分布配置
4. 配置设备级 Linalg 分块选项
5. 执行设备级分块和融合过程



TileAndDistributeToWorkgroupsPass::runOnOperation() 函数流程图

设备级分块过程实现

1. 循环分布方法

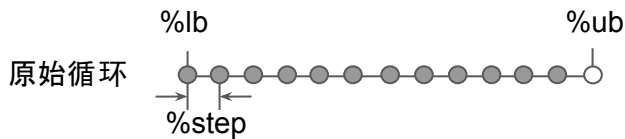
(1) Cyclic 循环分布策略

```
enum class DistributionMethod {  
    Cyclic = 0,  
    CyclicNumProcsGeNumIters = 1,  
    CyclicNumProcsEqNumIters = 2,  
    None = 3  
};
```

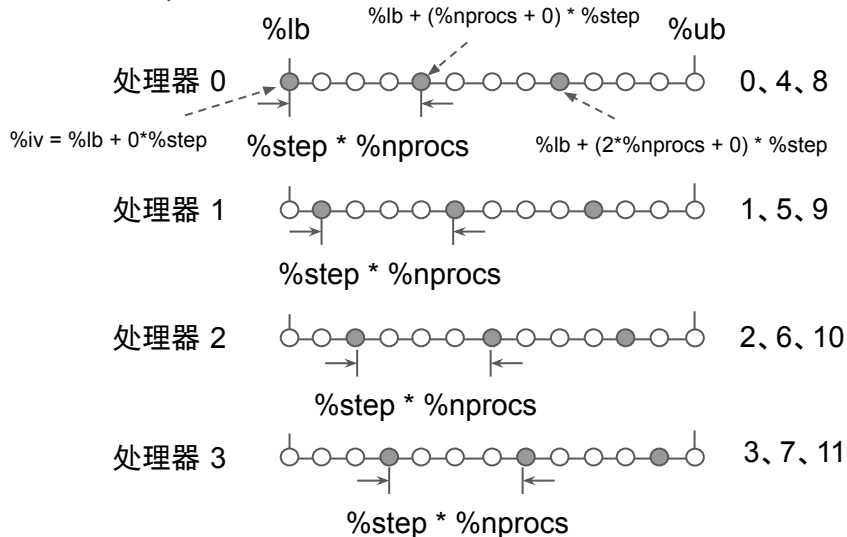
```
scf.parallel (%iv) = (%lb) to (%ub) step (%step) { // 循环体 }
```

↓ Cyclic 循环分布策略

```
scf.parallel (%iv) = (%lb + %procid * %step) to (%ub) step (%step * %nprocs) { // 循环体 }
```



↓ Cyclic 循环分布策略



设备级分块过程实现

1. 循环分布方法

(2)CyclicNumProcsGeNumIters 循环分布策略

```
enum class DistributionMethod {  
    Cyclic = 0,  
    CyclicNumProcsGeNumIters = 1,  
    CyclicNumProcsEqNumIters = 2,  
    None = 3  
};
```

```
scf.parallel (%iv) = (%lb) to (%ub) step (%step) { // 循环体 }
```

↓ CyclicNumProcsGeNumIters 循环分布策略

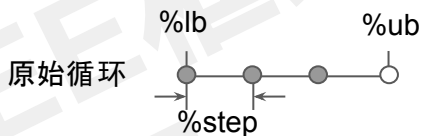
```
%iv = %lb + %proclid * %step
```

```
%cond = arith.cmpi "slt", %iv, %ub
```

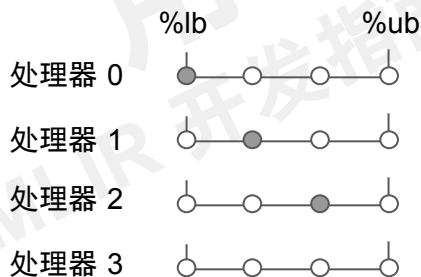
```
scf.if %cond {
```

```
    // 当 %cond 条件满足时执行计算任务
```

```
}
```



↓ CyclicNumProcsGeNumIters 循环分布策略



处理器 0: $\%iv = 0 + 0 * \%step = 0$ 且 $0 < 3$, 因此, 执行计算任务;

处理器 1: $\%iv = 0 + 1 * \%step = 1$ 且 $1 < 3$, 因此, 执行计算任务;

处理器 2: $\%iv = 0 + 2 * \%step = 2$ 且 $2 < 3$, 因此, 执行计算任务;

处理器 3: $\%iv = 0 + 3 * \%step = 3$ 且 $3 == 3$, 因此, 不执行计算任务。

设备级分块过程实现

```
scf.parallel (%iv) = (%lb) to (%ub) step (%step) { A[%iv] = B[%iv] + C[%iv] }
```

↓ CyclicNumProcsEqNumIters 循环分布策略

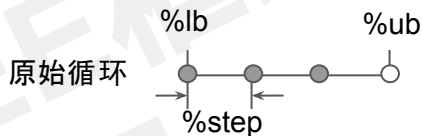
```
%iv = %lb + %procl * %step
```

```
A[%iv] = B[%iv] + C[%iv]
```

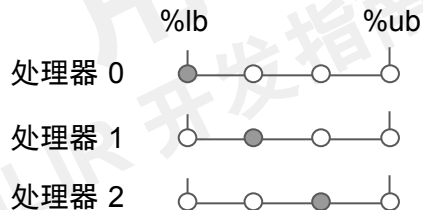
1. 循环分布方法

(3) CyclicNumProcsEqNumIters 循环分布策略

```
enum class DistributionMethod {  
    Cyclic = 0,  
    CyclicNumProcsEqNumIters = 1,  
    CyclicNumProcsEqNumIters = 2,  
    None = 3  
};
```



↓ CyclicNumProcsEqNumIters 循环分布策略



处理器 0: $\%iv = 0 + 0 * \%step = 0$, 因此, 执行计算任务 $A[0] = B[0] + C[0]$;

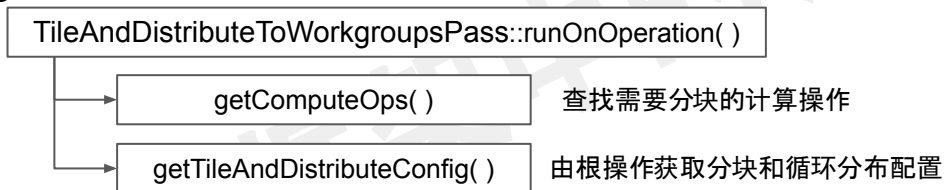
处理器 1: $\%iv = 0 + 1 * \%step = 1$, 因此, 执行计算任务 $A[1] = B[1] + C[1]$;

处理器 2: $\%iv = 0 + 2 * \%step = 2$, 因此, 执行计算任务 $A[2] = B[2] + C[1]$ 。

设备级分块过程实现

2. 获取计算操作

3. 获取分块和循环分布配置



计算操作集合 `computeOps` 是后续的分块和融合过程需要处理的操作集合。

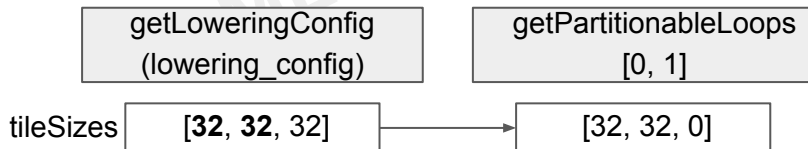
根操作是在 `computeOps` 集合中从后向前 查找得到的第一个具有 递降配置属性的 计算操作。

```
%6 = linalg.fill {lowering_config = #iree_codegen.lowering_config<tile_sizes = [[32, 32, 32]]>} ins(%cst : f16)
```

```
outs(%5 : tensor<512x512xf16>) -> tensor<512x512xf16>
```

```
%7 = linalg.matmul {lowering_config = #iree_codegen.lowering_config<tile_sizes = [[32, 32, 32]]>} ins(%3, %4 :
```

```
tensor<512x512xf16>, tensor<512x512xf16>) outs(%6 : tensor<512x512xf16>) -> tensor<512x512xf16>
```



`linalg.matmul` 操作的设备级分块大小

设备级分块过程实现

4. 配置设备级 Linalg 分块选项

(1) 设置循环分布选项

- `setDistributionOptions( )` 方法

(2) 设置循环分块类型

- `setLoopType( )` 方法

(3) 设置分块大小计算回调函数

- `setTileSizeComputationFunction( )` 方法

`TileAndDistributeToWorkgroupsPass::runOnOperation( )`

.....

`linalg::LinalgTilingOptions( )`

LinalgTilingOptions

distribution

loopType

tileSizeComputationFunction

interchangeVector

.....

LinalgLoopDistributionOptions

procInfo

```
enum class LinalgTilingLoopType {  
  Loops = 0,  
  AffineLoops = 1,  
  ParallelLoops = 2  
};
```

```
using TileSizeComputationFunction =  
  std::function<SmallVector<Value, 4>(OpBuilder &,  
    Operation *)>;
```

```
auto tileSizeFn = [&](OpBuilder &builder,  
  Operation *op) -> SmallVector<Value> {  
  return llvm::map_to_vector(tileSizes, [&](int64_t ts) -> Value {  
    return builder.create<arith::ConstantIndexOp>(op->getLoc(), ts);  
  });  
};
```

设备级分块过程实现

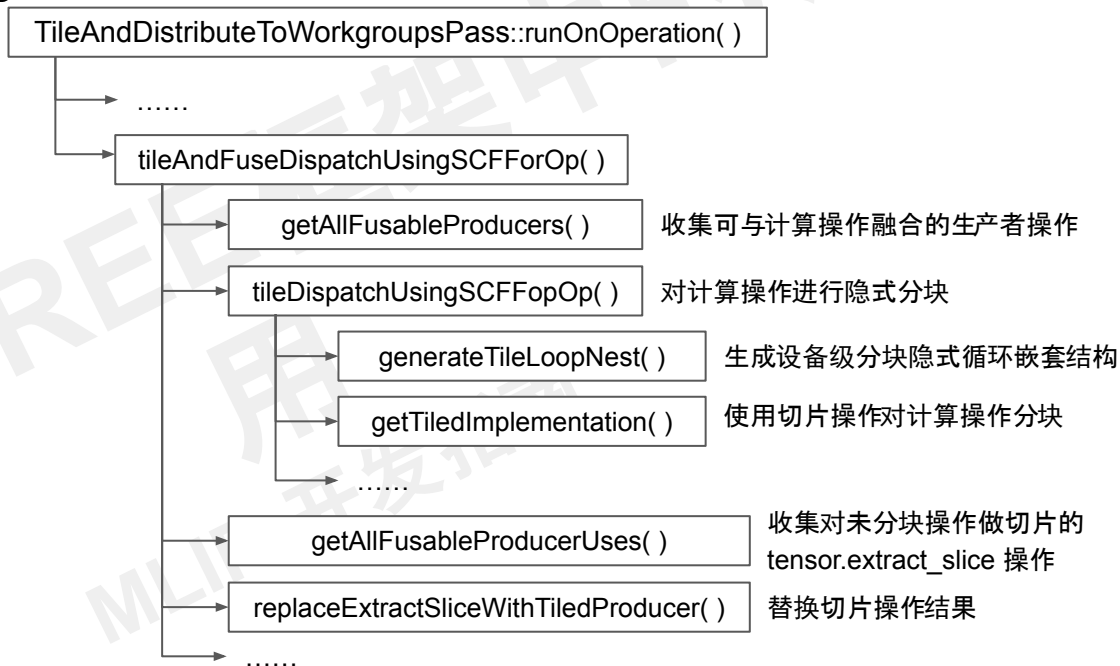
5. 执行设备级分块和融合过程

(1) 收集可融合生产者操作

(2) 生成设备级分块循环并执行分块

(3) 收集切片操作

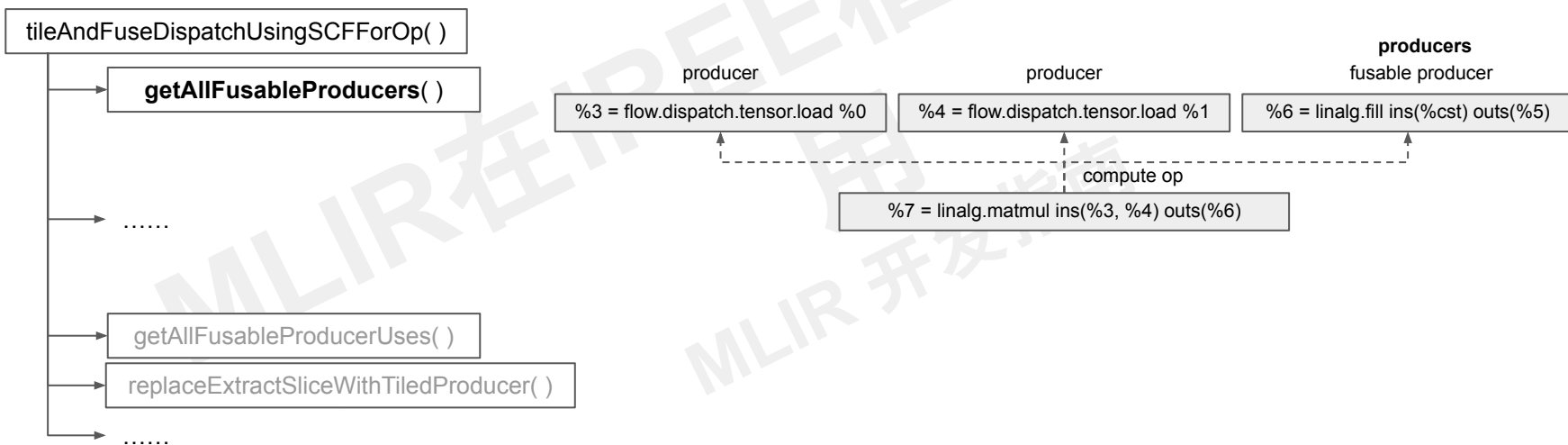
(4) 替换切片操作结果



设备级分块过程实现

5. 执行设备级分块和融合过程

(1) 收集可融合生产者操作



设备级分块过程实现

5. 执行设备级分块和融合过程

(1) 收集生产者操作

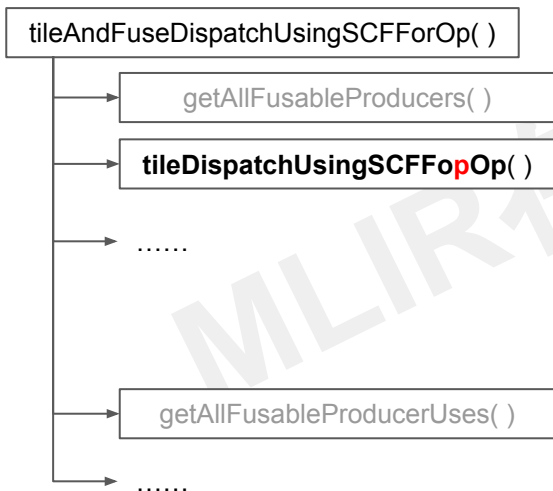
(2) 生成设备级分块循环并进行分块

1) 获取计算操作迭代域和并行循环范围向量

2) 计算工作组处理器信息

3) 生成设备级分块隐式循环嵌套结构

4) 对计算操作进行分块



设备级分块过程实现

5. 执行设备级分块和融合过程 1) 获取计算操作迭代域和并行循环范围向量

(1) 收集生产者操作

(2) 生成设备级分块循环并进行分块

```
linalg.matmul {lowering_config = #iree_codegen.lowering_config<tile_sizes = [[32, 32, 32]]>} ins(%3, %4 : tensor<512x512xf16>, tensor<512x512xf16>) outs(%6 : tensor<512x512xf16>) -> tensor<512x512xf16>
```

```
struct Range {  
    OpFoldResult offset;  
    OpFoldResult size;  
    OpFoldResult stride;  
};
```

getIterationDomain()

getLoopIteratorTypes()

tileSizesFn()

(s1 - s0) ceildiv (s2 * s3)

(size - offset) / (stride * tileSizes)

tileAndFuseDispatchUsingSCFForOp()

getAllFusableProducers()

tileDispatchUsingSCFFopOp()

.....

getAllFusableProducerUses()

.....

	M	N	K
迭代域向量:	[[0, 512, 1],	[0, 512, 1],	[0, 512, 1]]

迭代器类型数组: [[0, parallel], {1, parallel}, {2, reduction}]

分块大小向量: 32 32 0

分块数量: 16 16

parallelLoopRanges: [[0, 512, 1], [0, 512, 1]]

options.distribution->procInfo(rewriter, loc, parallelLoopRanges)

设备级分块过程实现

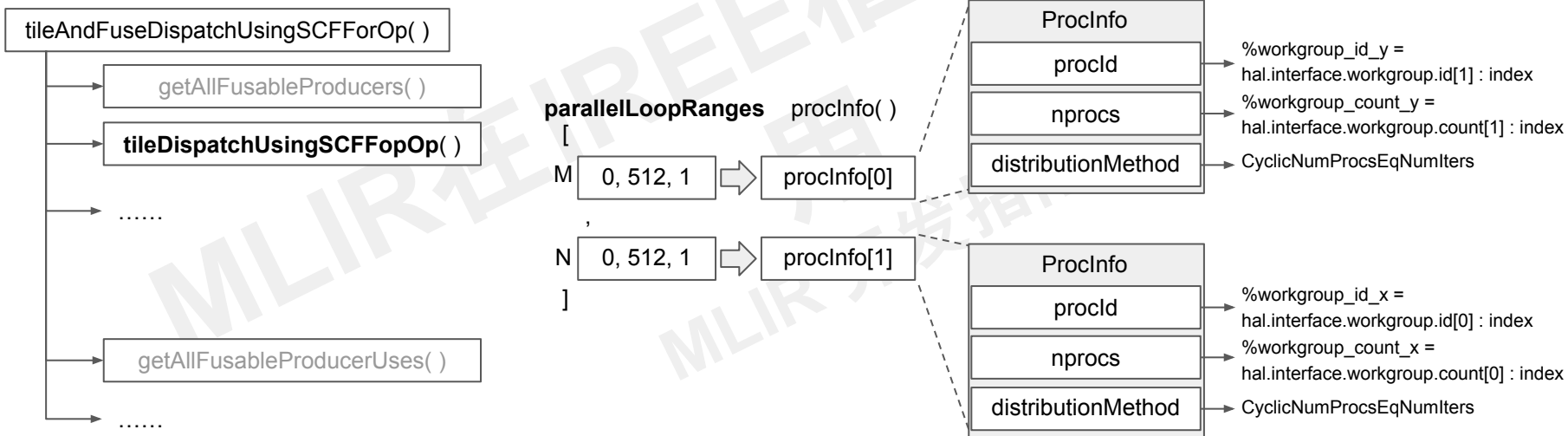
5. 执行设备级分块和融合过程

(1) 收集生产者操作

(2) 生成设备级分块循环并进行分块

2) 计算工作组处理器信息

计算处理器信息 lambda 函数在 `getIREELinalgLoopDistributionOptions()` 函数中定义。



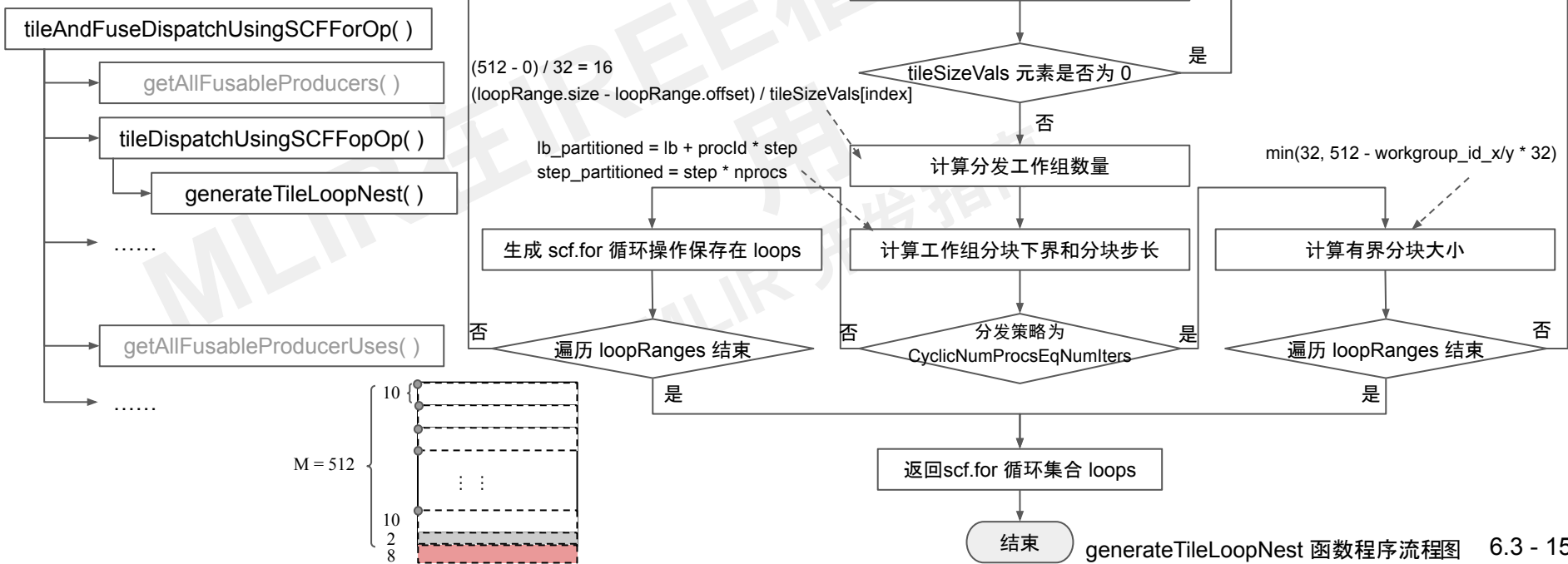
`linalg.matmul` 操作的处理器(工作组)信息向量

设备级分块过程实现

5. 执行设备级分块和融合过程 3)生成设备级分块隐式循环嵌套结构

(1) 收集生产者操作

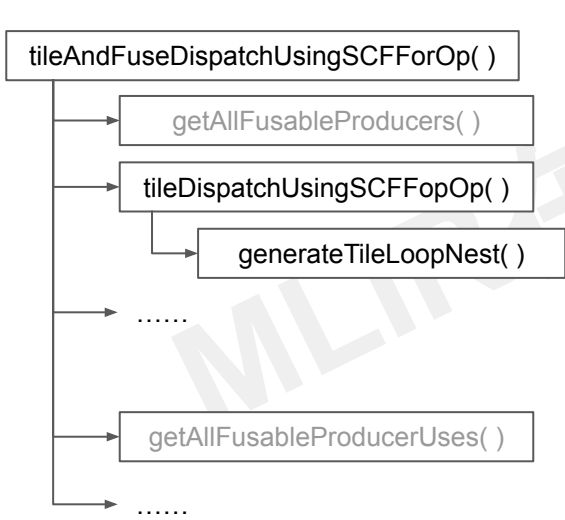
(2) 生成设备级分块循环并进行分块



设备级分块过程实现

5. 执行设备级分块和融合过程 3)生成设备级分块隐式循环嵌套结构

- (1)收集生产者操作
- (2)生成设备级分块循环并进行分块



AfineMap	minMap	numIterationsMap	offsetMap	stepMap
表达计算	$\min(\text{tileSize}, \text{ub} - \text{iv})$	$(\text{loopRange.size} - \text{loopRange.offset}) / \text{tileSize}$	$\text{lb} + \text{procId} * \text{step}$	$\text{step} * \text{nprocs}$
含义	计算有界分块大小 (sizes)	计算分发工作组数量	计算工作组分块下界(offsets)	计算分块步长
仿射映射	$(\text{d0})[\text{s0}, \text{s1}] \rightarrow (\text{s0}, \text{s1} - \text{d0})$	$(\text{])[s0}, \text{s1}, \text{s2}] \rightarrow ((\text{s1} - \text{s0}) \text{ ceildiv } \text{s2})$	$(\text{])[s0}, \text{s1}, \text{s2}] \rightarrow (\text{s0} + \text{s1} * \text{s2})$	$(\text{])[s0}, \text{s1}] \rightarrow (\text{s0} * \text{s1})$

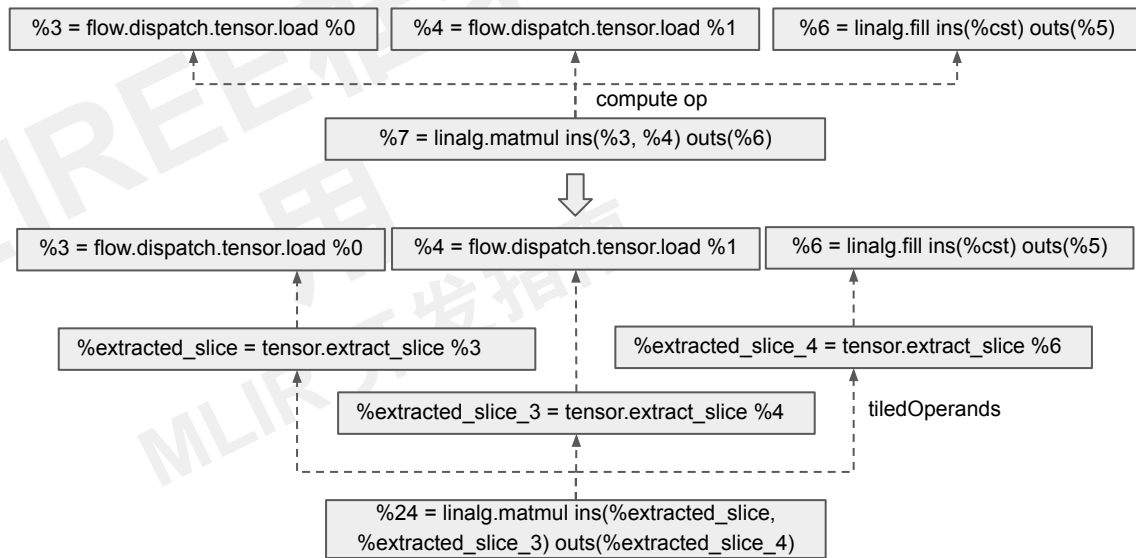
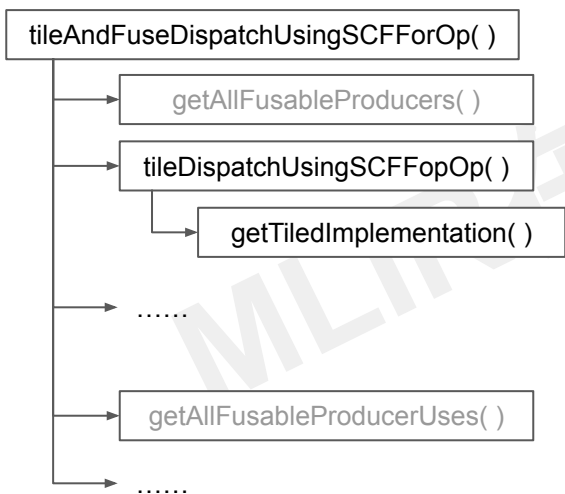
设备级分块过程实现

generateTileLoopNest() 返回值 offsets, sizes

5. 执行设备级分块和融合过程 4) 对计算操作进行分块

(1) 收集生产者操作

(2) 生成设备级分块循环并进行分块



ins(... : tensor<512x512xf16>, tensor<512x512xf16>)
outs(... : tensor<512x512xf16>)

分块前



ins(... : tensor<32x512xf16>, tensor<512x32xf16>)
outs(... : tensor<32x32xf16>)

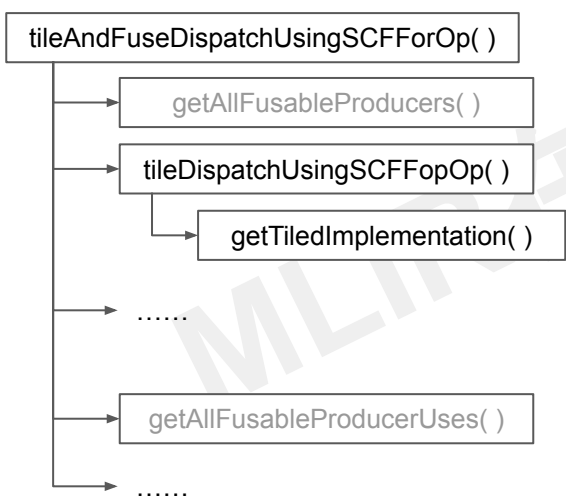
分块后

设备级分块过程实现

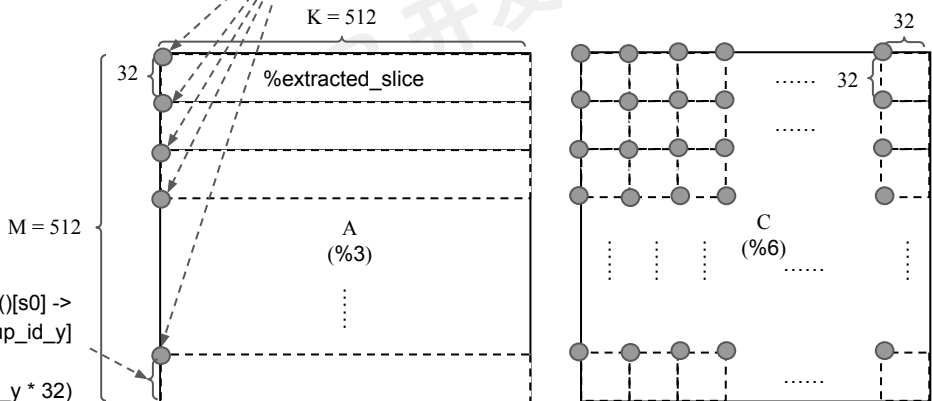
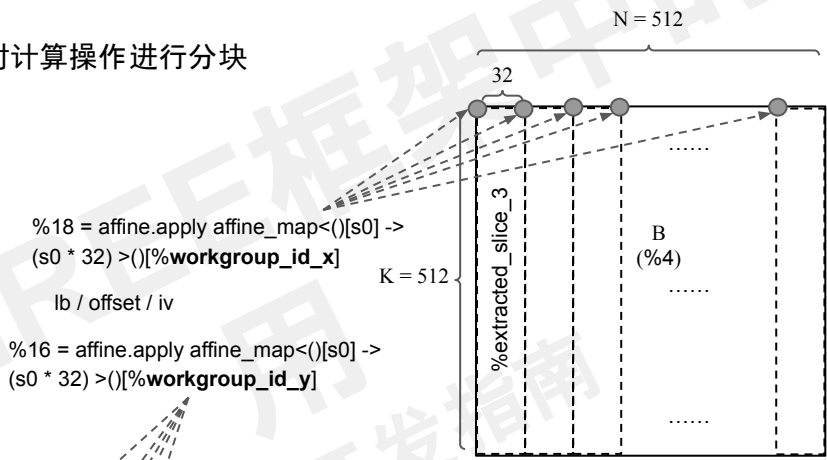
5. 执行设备级分块和融合过程 4) 对计算操作进行分块

(1) 收集生产者操作

(2) 生成设备级分块循环并进行分块



generateTileLoopNest() 返回值 offsets, sizes



$\%20 = \text{affine.apply affine_map}(<()[s0] \rightarrow (s0 * 32) >())[\%workgroup_id_y]$

$\%22 = \text{affine.apply affine_map}(<()[s0] \rightarrow (s0 * 32) >())[\%workgroup_id_x]$

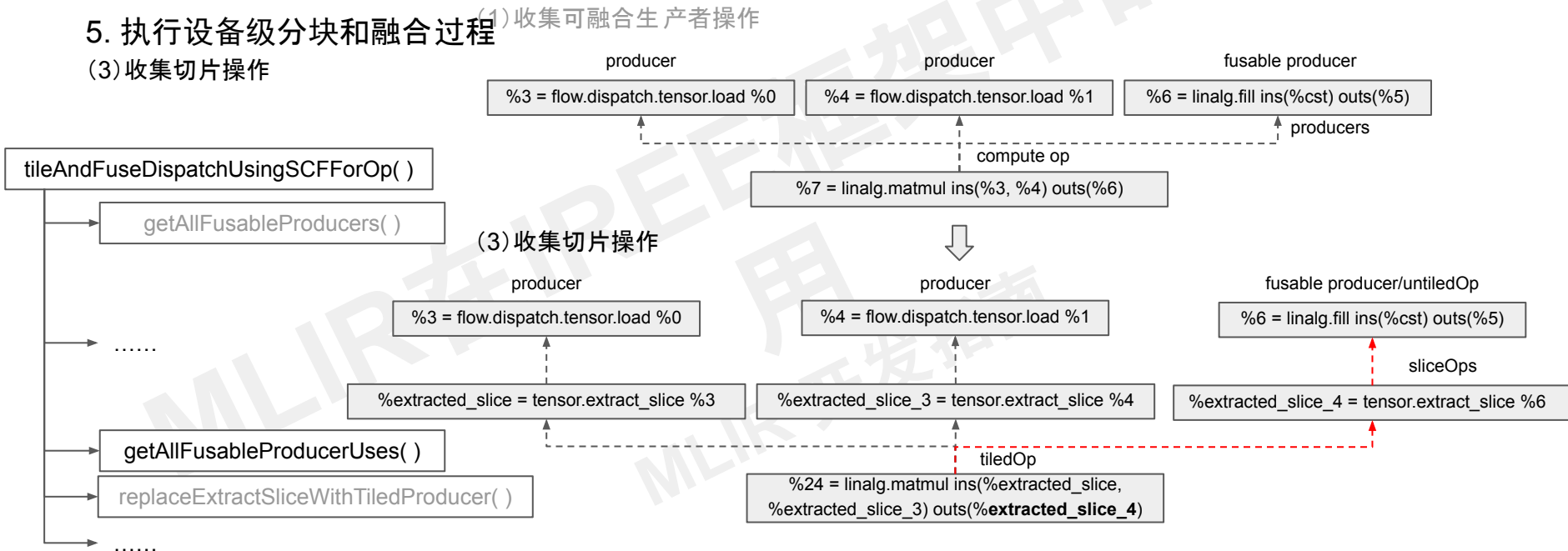
$\%10 = \text{affine.min affine_map}(<()[s0] \rightarrow (32, s0 * -32 + 512) >())[\%workgroup_id_y]$

bounded size = $\min(32, 512 - \text{workgroup_id_y} * 32)$

设备级分块过程实现

5. 执行设备级分块和融合过程

(3) 收集切片操作

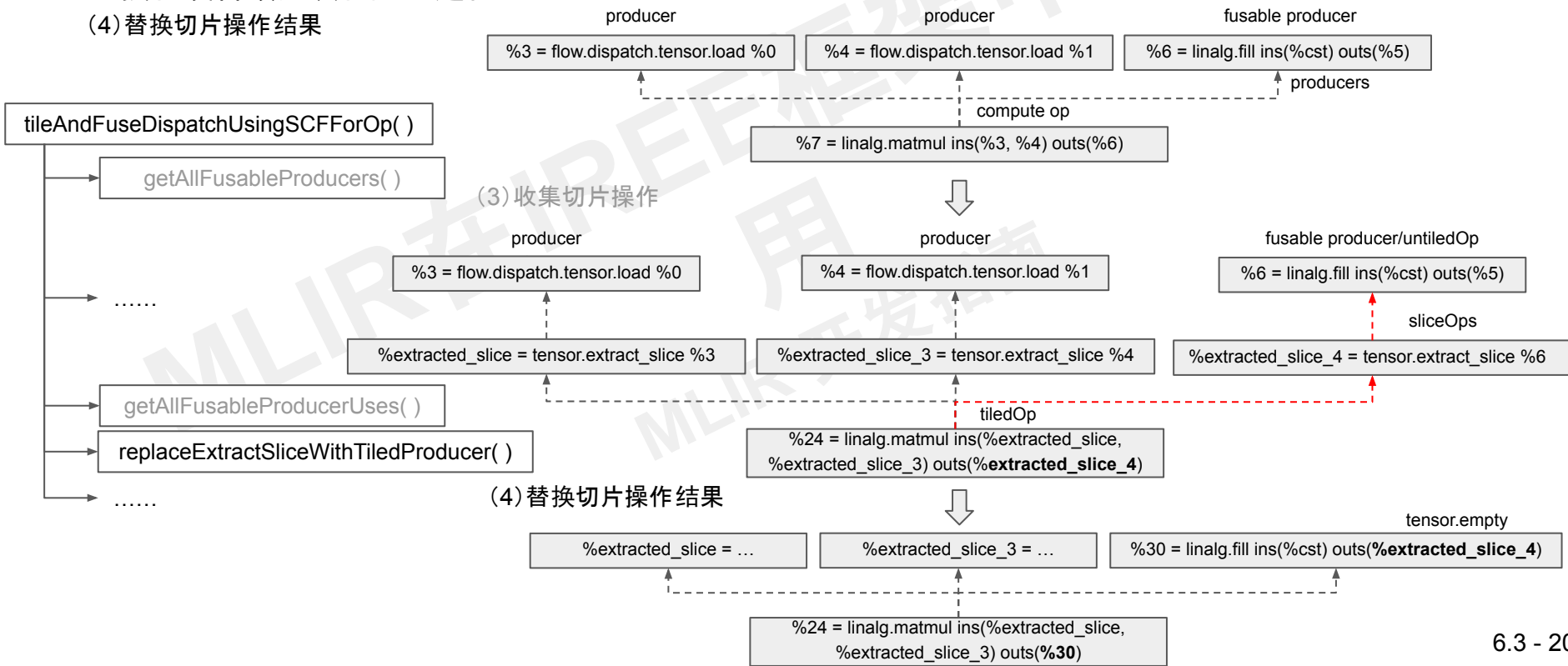


设备级分块过程实现

5. 执行设备级分块和融合过程

(1) 收集可融合生产者操作

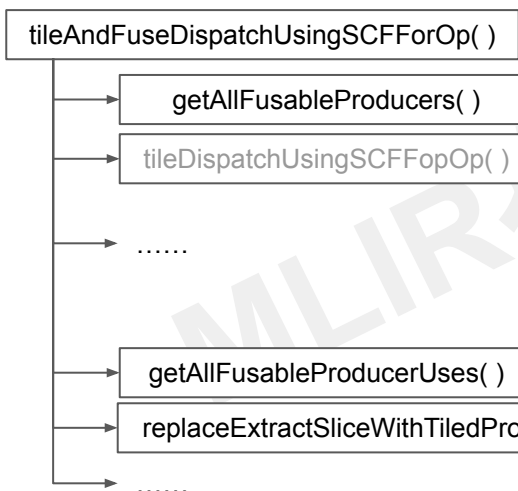
(4) 替换切片操作结果



设备级分块过程实现

融合过程总结

5. 执行设备级分块和融合过程



```
%C = linalg.fill ... // 未分块生产者操作, 生成张量 C
%slice_A = tensor.extract_slice %A ... // 从 A 中提取切片
%slice_B = tensor.extract_slice %B ... // 从 B 中提取切片
%slice_C = tensor.extract_slice %C ... // 从 C 中提取切片
%D = linalg.matmul %slice_A, %slice_B, %slice_C... // 已分块消费者操作, 使用切片 进行计算
```

↓ tileAndFuseDispatchUsingSCFForOp

```
%1 = tensor.empty ... // 未分块 tensor.empty
%slice_1 = tensor.extract_slice %1 ... // 从 tensor.empty 结果操作数中提取切片
%slice_A = tensor.extract_slice %A ... // 从 A 中提取切片
%slice_B = tensor.extract_slice %B ... // 从 B 中提取切片
%slice_C = tensor.extract_slice %C ... // 从 B 中提取切片
%C1 = linalg.fill %slice_1... // 已分块生产者操作, 生成张量 C1
%D = linalg.matmul %slice_A, %slice_B, %C1... // 已分块消费者操作, 使用切片 进行计算
```

↓ SwapExtractSliceWithDispatchTensorLoad
SwapExtractSliceWithTensorEmpty

```
%1_1 = tensor.empty ... // 已分块 tensor.empty
%A1 = flow.dispatch.tensor.load ... // 已分块 flow.dispatch.tensor.load
%B1 = flow.dispatch.tensor.load ... // 已分块 flow.dispatch.tensor.load
%C1 = linalg.fill %1_1... // 已分块生产者操作, 生成张量 C1
%D = linalg.matmul %A1, %B1, %C1... // 已分块消费者操作, 使用切片 进行计算
```

设备级分块和融合过程实现总结

```
%7 = linalg.matmul {lowering_config = #iree_codegen.lowering_config<tile_sizes = [[32, 32, 32]]>} ins(%3, %4 :  
tensor<512x512xf16>, tensor<512x512xf16>) outs(%6 : tensor<512x512xf16>) -> tensor<512x512xf16>
```



分块大小向量: [32, 32, 32]



可切分循环

分块大小向量: [32, 32, 0]



迭代域向量: [[0, 512, 1], [0, 512, 1], [0, 512, 1]]



迭代器类型

parallelLoopRanges: [[0, 512, 1], [0, 512, 1]]



procInfo[0]

procInfo[1]

ProcInfo

procId

nprocs

distributionMethod